

Midterm Progress Report

Project: Building Reasoning LLMs from Scratch using Chain-of-Thought
Reasoning (DeepSeek R1)

WIDS'25 - Analytics Club, IIT Bombay

Name: Meet Agrawal

Contents

1	Introduction	3
2	The Reasoning Framework (Week 1)	3
2.1	Chain-of-Thought (CoT) Reasoning	3
2.2	Verifiers and Reward Models	3
3	Reinforcement Learning Foundations	4
3.1	The Agent-Environment Interface	4
3.2	The Multi-Armed Bandit Problem	4
4	Markov Decision Processes (MDPs)	4
4.1	Formal Definition	4
5	Week 2: Solving the MDP	5
5.1	Value Functions	5
5.2	Monte Carlo (MC) Methods	5
5.3	Temporal Difference (TD) Learning	5
6	Architecture Deep Dive: Transformers	6
6.1	Attention Mechanism	6
7	Conclusion	6
8	Week 3: Advanced Reasoning Frameworks	7
8.1	From Next-Token Prediction to Policy Learning	7
8.2	Detailed Comparison: Outcome Supervision vs Process Supervision	8
8.3	Credit Assignment and Temporal-difference Analogues	8
8.4	Verifiers as Critics: Roles and Failure Modes	9
8.5	Relative Optimization and GRPO Intuition	9
9	Week 4: System-Level Design and Research Perspectives	10
9.1	Reasoning as a Latent Trajectory vs. Explicit CoT	10
9.2	Scaling Reasoning: Efficiency and Pruning Strategies	10
9.3	Robust Evaluation Beyond Final Accuracy	11
9.4	Failure Modes and Defensive Measures	11
9.5	Ethical and Alignment Considerations	12
10	Synthesis: How Weeks 3 & 4 Extend the Midterm	12
11	Conclusion and Suggested Next Steps	12

1 Introduction

This report documents the progress made during the first two weeks of the Winter in Data Science (WIDS) 2025 project. The project’s objective is to deconstruct and implement ”Reasoning” Large Language Models (LLMs), drawing inspiration from architectures like DeepSeek R1. Key topics covered include Chain-of-Thought (CoT) reasoning, Verifier models, Markov Decision Processes (MDPs), etc.

Artificial Intelligence is shifting from fast-pattern recognition toward slower-reasoning. LLM models like GPT-3 are strong at predicting text, they still find multi-step logical and mathematical reasoning challenging.

This project focuses on overcoming these limitations by combining Reinforcement Learning (RL) with Chain-of-Thought (CoT) prompting. The goal is to train models that can reason step by step, check their own logic, and correct mistakes along the way.

2 The Reasoning Framework (Week 1)

2.1 Chain-of-Thought (CoT) Reasoning

Standard LLMs map input prompts directly to output responses. In contrast, CoT reasoning introduces intermediate **thoughts**.

- **Concept:** Instead of asking the model for the answer directly ($Input \rightarrow Answer$), we prompt it to generate a reasoning path ($Input \rightarrow Reasoning \rightarrow Answer$).
- **Impact:** This improves performance on reasoning and arithmetic tasks by decomposing complex problems into sub-problems.

2.2 Verifiers and Reward Models

The concept of **Verifier** was introduced in week 1. In the context of DeepSeek R1 and similar architectures, generating a thought is one part of whole process and evaluating it is another.

The Verifier acts as a **Reward Model** in the RL framework. It scores the reasoning steps, providing the feedback signal necessary for the model to learn. We reviewed two types of feedback:

1. **Outcome Supervision:** Rewarding the model only if the final answer is correct.
2. **Process Supervision:** Rewarding each step of the reasoning chain (Process Reward Models). This is generally more effective for complex reasoning tasks.

3 Reinforcement Learning Foundations

3.1 The Agent-Environment Interface

We defined the RL loop in which the **Agent** (our LLM) interacts with an **Environment** (the context/prompt) at discrete time steps t .

- **State** (S_t): The current situation (e.g., the text generated so far).
- **Action** (A_t): The decision made by the agent (e.g., predicting the next token).
- **Reward** (R_{t+1}): A scalar feedback signal indicating how good the action was.

The goal of the agent is to maximize the expected cumulative reward, denoted as the return G_t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1)$$

where γ is the discount factor ($0 \leq \gamma \leq 1$).

3.2 The Multi-Armed Bandit Problem

- **The Problem:** The agent must choose between k options (arms). The agent does not know the probability distribution of rewards for the arms.
- **Exploration vs. Exploitation:**
 - *Exploitation:* Choosing the arm with the highest estimated value to maximize immediate reward.
 - *Exploration:* Choosing a sub-optimal arm to gather data, possibly discovering a better arm.
- **ϵ -Greedy Algorithm:** In this algorithm, the agent explores with probability ϵ and exploits with probability $1 - \epsilon$.

4 Markov Decision Processes (MDPs)

4.1 Formal Definition

We formalized sequential decision-making using Markov Decision Processes. An MDP is defined by the tuple (S, A, P, R, γ) . The core assumption is the **Markov Property**:

$$P(S_{t+1}|S_t, A_t) = P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots) \quad (2)$$

This implies that the current state contains all necessary information about the history to predict the future.

5 Week 2: Solving the MDP

In Week 2, we focused on the algorithms used to solve MDPs.

5.1 Value Functions

To choose the best action, we must estimate the "value" of being in a state.

- **State-Value Function** $V_\pi(s)$: How good is it to be in state s ?
- **Action-Value Function** $Q_\pi(s, a)$: How good is it to take action a in state s ?

The relationship between these values is defined by the **Bellman Expectation Equation**:

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V_\pi(s')] \quad (3)$$

This recursive equation allows us to compute values using Dynamic Programming if the model dynamics (p) are known.

5.2 Monte Carlo (MC) Methods

In the case of LLMs, we don't know the full transition dynamics of the environment. Monte Carlo methods solve this by learning from **experience**.

- **Mechanism**: The agent plays out full episodes. At the end of an episode, it calculates the return G_t and updates the value estimate $V(S_t)$ towards G_t .
- **Limitation**: MC methods can only update after an episode terminates.

5.3 Temporal Difference (TD) Learning

TD Learning is a central concept for our project. It combines the benefits of Dynamic Programming (bootstrapping) and Monte Carlo (sampling).

- **The Update Rule (TD(0))**:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (4)$$

- **Significance**: The agent updates its estimate based on an estimate of the next state ($V(S_{t+1})$). This allows the model to learn *during* the episode, step-by-step.
- **Application to LLMs**: This is crucial for training reasoning models, as we want the model to realize it made a logical error immediately, rather than waiting until the entire paragraph is generated.

6 Architecture Deep Dive: Transformers

(Based on supplementary resources shared on Transformer Architectures)

Reviewed the underlying architecture of LLMs: The Transformer.

6.1 Attention Mechanism

The core innovation of the Transformer is the Self-Attention mechanism, which allows the model to weigh the importance of different words in a sequence relative to each other.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

Understanding this flow of information is vital when we later implement RL, as the policy $\pi(a|s)$ is parameterized by these attention weights.

7 Conclusion

1. Reasoning can be modeled as a sequential decision process (MDP).
2. Verifiers act as the source of truth (Reward) in this process.
3. TD Learning allows for efficient updates during the reasoning generation.

Abstract

This document continues the midterm progress report for the DeepSeek R1 project and documents the theoretical work undertaken in Weeks 3 and 4. It develops advanced perspectives on reasoning as sequential decision-making, examines supervision paradigms, analyses credit assignment in long reasoning trajectories, and surveys system-level constraints, evaluation strategies, and alignment considerations relevant to reasoning-capable LLMs. The present report builds directly upon the submitted midterm report and the project overview. Footnotes link to the original midterm and overview materials for continuity.^{1 2}

The midterm report summarized foundational concepts established in Weeks 1 and 2: Chain-of-Thought (CoT) reasoning, verifiers as reward models, the agent-environment interface in reinforcement learning, multi-armed bandits, Markov Decision Processes (MDPs), and primary algorithmic families such as Monte Carlo and Temporal Difference learning. The present continuation (Weeks 3 and 4) moves beyond those foundational ideas and develops the theoretical scaffolding required to reason about *how* reasoning-capable language models should be conceptualized, trained, and evaluated at the system level. The objective is not to present novel experimental results, but rather to provide a careful synthesis of modern perspectives that logically extend the earlier weeks’ material and that adhere to the project’s stated goals and mentor guidance.

The central thesis of Weeks 3 and 4 is that the challenge of reliable multi-step reasoning requires reinterpreting intermediate model outputs as *policy trajectories* amenable to structured supervision and critic-based evaluation. This reframing motivates distinct training objectives, evaluation metrics, and system design choices that are qualitatively different from the next-token prediction paradigm that underlies conventional language models.

8 Week 3: Advanced Reasoning Frameworks

Week 3 concentrated on theoretical reframing, comparative analysis of supervision paradigms, mechanisms for reward propagation in long reasoning sequences, and the conceptual role of verifier modules. The following subsections present in-depth expositions of these themes.

8.1 From Next-Token Prediction to Policy Learning

Traditional LLM training optimizes a maximum likelihood objective over token sequences. This objective encourages fluent continuation but does not directly encode desiderata such as logical soundness, stepwise consistency, or long-horizon correctness. In contrast, by viewing each token or intermediate sentence as an *action* chosen by a policy, we can leverage reinforcement learning concepts to align generation behavior with structured goals.

¹See Midterm Progress Report submitted earlier: :contentReference[oaicite:2]index=2.

²Project overview and mentor guidelines

Under this view, an input prompt defines an initial state; intermediate reasoning utterances are actions that transition the model into new states; and verifiers or external metrics produce reward signals that guide policy improvement. The immediate implication is that training objectives must account for long-term returns rather than merely immediate token likelihood. This motivates the adoption (conceptually) of policy optimization techniques and critic-guided learning, where the critic (verifier) assigns scalar feedback reflecting logical fidelity and coherence across steps.

The theoretical reframing illuminates why many reasoning failures persist when applying only supervised fine-tuning or prompt engineering: such strategies do not provide long-horizon reinforcement for maintaining internally consistent chains of inference.

8.2 Detailed Comparison: Outcome Supervision vs Process Supervision

Week 1 introduced the distinction between outcome supervision (reward only for correct final answers) and process supervision (rewarding intermediate steps). Week 3 expanded this into a rigorous comparative analysis. Outcome supervision is attractive due to its simplicity: correctness at the end can be binary and easily verified for some tasks. However, it suffers from two fundamental problems. First, the reward signal is *sparse* and *delayed*, exacerbating credit assignment issues. Second, outcome-only incentives can be gamed: models may learn to produce superficially plausible final answers without constructing valid intermediate logic (a phenomenon we term *shortcut reasoning*).

Process supervision combats these problems by providing dense, step-wise feedback. Verifiers that evaluate each intermediate step allow the system to localize errors, reduce variance in learning updates, and reinforce valid subroutines of reasoning. Conceptually, process supervision approximates a shaping reward in RL: it helps the learner discover productive trajectories more efficiently.

Week 3 emphasized that process supervision is not a panacea; its design must avoid introducing spurious biases. For example, overfitting to a verifier’s heuristics can lead to brittle reasoning that satisfies the verifier but fails under perturbation. Thus, much of the week’s theoretical work focused on the *design space* of verifiers and on criteria for robust process supervision.

8.3 Credit Assignment and Temporal-difference Analogues

Long reasoning chains raise difficult credit-assignment problems: when a final output is incorrect, how should blame be apportioned to early versus late steps? Week 3 analyzed this through analogies to Temporal Difference (TD) and Monte Carlo (MC) methods described in Week 2. MC approaches evaluate entire episodes, offering unbiased estimates of return but suffering from high variance. TD methods bootstrap from value estimates of successor states to provide more stable updates.

Mapping this to reasoning, TD-style updates correspond to adjusting the estimated

quality of intermediate reasoning states based on the estimated quality of subsequent states (which may themselves be estimated by a verifier or learned value function). This perspective motivates hybrid learning rules in reasoning systems that blend immediate local evaluations with bootstrapped future evaluations to achieve tractable learning dynamics.

The conceptual exercise in Week 3 paid particular attention to the problem of *error propagation*. Erroneous early steps can compound, making final outcomes appear plausible while being logically unsound. A theoretical remedy—discussed at length—was to design verifiers that estimate not only step validity but also the *expected future utility* of a given intermediate step, enabling more informative corrections.

8.4 Verifiers as Critics: Roles and Failure Modes

Verifiers occupy a central role in the reasoning pipeline: they are responsible for judging the quality of intermediate steps and thereby shaping policy updates. Week 3 drew an explicit parallel between verifiers and critics in actor-critic RL architectures. As critics, verifiers produce scalar feedback that the policy uses for improvement. However, unlike classical RL critics trained against ground-truth rewards, verifiers for reasoning must often rely on heuristics, auxiliary tasks, or weaker supervision.

This dual role produces several failure modes that were studied theoretically:

- **Incentive misalignment:** A verifier may reward superficial stylistic traits or token-level cues that correlate with correctness in the training set but do not generalize.
- **Reward hacking:** The policy may discover strategies that maximize verifier scores while producing logically incorrect content that nevertheless satisfies the verifier’s checks.
- **Calibration issues:** Verifier judgments can be overconfident or poorly calibrated, causing unstable policy updates.

Week 3 concluded that robust verifier design requires a mixture of approaches: adversarial testing, ensemble verification, and cross-checking via independent reasoning paths, all considered at a conceptual level without implementation.

8.5 Relative Optimization and GRPO Intuition

Group Relative Policy Optimization (GRPO) is an optimization intuition that surfaced in project readings and discussions. Conceptually, GRPO prefers evaluating candidate trajectories relative to one another rather than relying solely on absolute reward estimates. This can mitigate scale and calibration problems in verifiers: by comparing alternatives, the system can prefer trajectories with demonstrably better properties even when absolute scores are noisy.

The Week 3 material elaborated on why relative comparisons can reduce the propensity for reward hacking. If the verifier’s absolute score is unreliable, the relative ordering of well-formed vs. poorly formed trajectories still provides a useful learning signal. The group-relative perspective thus becomes a pragmatic design principle in the absence of an oracle-level verifier.

9 Week 4: System-Level Design and Research Perspectives

Week 4 broadened the scope from algorithmic primitives to system-level constraints and broader research questions. The discussions were interdisciplinary, connecting scalability, evaluation, interpretability, and alignment considerations. The following subsections synthesize those themes.

9.1 Reasoning as a Latent Trajectory vs. Explicit CoT

A key conceptual distinction developed in Week 4 is between reasoning that is explicit (visible chain-of-thought output) and reasoning that is latent (internal representations guiding decisions). Explicit chains are invaluable for interpretability and human oversight; latent reasoning may nonetheless produce robust decisions without exposing intermediate steps. The trade-off is subtle: forcing explicit chains helps diagnosis but may increase computational cost and provide additional surface for gaming.

Week 4 argued that practical systems can combine both approaches: explicit chains for high-stakes decisions where transparency matters, and latent representations for efficiency in lower-risk settings. Moreover, the report outlined theoretical criteria for when explicit explanation is necessary—e.g., when decisions are complex, high-consequence, or require external verification.

9.2 Scaling Reasoning: Efficiency and Pruning Strategies

Long chains of reasoning impose computational burdens. Unconstrained expansion of chain length can lead to prohibitive computational and annotation costs. Week 4 conceptually explored mitigation strategies:

- **Selective Reasoning:** Trigger deeper chains only for items that exceed a confidence threshold, thereby focusing resources.
- **Verifier-Guided Pruning:** Use lightweight verifiers to prune unpromising partial trajectories early in the process.
- **Hierarchical Reasoning:** Compose short, reliable subroutines that can be reused across tasks to reduce per-instance reasoning effort.

- **Cached Sub-derivations:** Maintain a repository of commonly used sub-steps or lemmas to avoid recomputation.

Each approach was discussed in terms of trade-offs between efficiency, correctness, and ease of verification. The emphasis was on principled, conceptual strategies rather than system engineering details.

9.3 Robust Evaluation Beyond Final Accuracy

Evaluation of reasoning systems cannot rely solely on final answer accuracy. Week 4 identified several essential axes of evaluation:

1. **Step Validity:** The fraction of intermediate steps that are logically valid.
2. **Consistency:** The degree to which the chain-of-thought maintains internal coherence.
3. **Robustness to Perturbations:** Sensitivity to minor changes in prompts or problem formulation.
4. **Calibrated Confidence:** Whether the model’s confidence estimates correspond to actual correctness probabilities.
5. **Generalization of Reasoning Patterns:** The ability to apply learned subroutines to novel problem instances.

The report elaborated conceptually on how these metrics can be approximated via human evaluation, verifier ensembles, and adversarial testing. It also emphasized that evaluation design must be tightly coupled to verifier construction to avoid circularity.

9.4 Failure Modes and Defensive Measures

Week 4 returned to failure modes but expanded the analysis to include systemic and social considerations. Key failure modes included:

- **Hallucinated Logic:** Confident but invalid intermediate steps that masquerade as reasoning.
- **Shortcut Solutions:** Solutions that exploit dataset quirks rather than demonstrating true reasoning.
- **Overfitting to Verifiers:** Policies that learn to satisfy specific verifier heuristics while reducing real logical capability.

For each mode, the report presented conceptual defensive measures: multi-view verification, adversarial example generation, human-in-the-loop checks for high-stakes outputs, and regular calibration of verifier thresholds.

9.5 Ethical and Alignment Considerations

The final portion of Week 4’s work emphasized that improved reasoning capability raises both potential benefits and risks. More powerful reasoning agents can assist in complex decision-making but also can be persuasive even when incorrect. The report articulates key alignment concerns:

- **Deceptive Plausibility:** A sophisticated reasoner may generate highly persuasive but false arguments.
- **Misplaced Trust:** Users may over-rely on explanations that appear plausible even when they are not vetted.
- **Weaponization Risks:** Advanced reasoning capabilities could be repurposed for creating manipulative content or automated decision-making that bypasses human judgment.

Week 4 advocated for conservative deployment practices, layered verification, and ongoing human oversight as conceptual safeguards, consistent with the project’s emphasis on verifier-driven validation.

10 Synthesis: How Weeks 3 & 4 Extend the Midterm

The midterm concentrated on establishing the foundations: what CoT is, why MDPs and TD learning matter, and how verifiers conceptually provide reward signals. Weeks 3 and 4 extend these foundations by:

- Reframing CoT outputs as policy trajectories subject to sequential decision-theoretic analysis.
- Developing a deeper conceptual taxonomy of supervision regimes and verifier roles.
- Analyzing credit assignment for long chains through TD and MC analogies.
- Providing system-level perspectives on scalability, evaluation, failure modes, and alignment.

This progression reflects a natural pedagogical arc: from definitions and primitives (Weeks 1–2) to a theoretical synthesis and system-level view (Weeks 3–4), in line with the project’s scope and mentor guidance.

11 Conclusion and Suggested Next Steps

Weeks 3 and 4 prepared a comprehensive theoretical foundation for potential experimental work. Although no coding or experiments were conducted post-midterm (as requested), the conceptual agenda suggests several next steps for future implementation:

1. **Design of verifier ensembles** to reduce single-verifier brittleness.
2. **Simulation studies** using synthetic tasks to test relative optimization heuristics like GRPO.
3. **Human evaluation protocols** to measure step validity at scale.
4. **Investigation of selective reasoning policies** to balance accuracy and compute cost.

These recommendations are framed as options for future phases, and are not reported as completed work. The present document is careful to claim only theoretical and conceptual progress consistent with the post-midterm timeline and the absence of coding work.